Feature Engineering (Author = Miroj Deep Bhandari)

Definition

- **Feature engineering** is the process of using **domain knowledge** to extract features or create features from raw data in order to improve the performance of machine learning algorithms.

Raw data collected from the real world is rarely in a form that can be directly fed into a machine learning algorithm. It must be cleaned, shaped, inferred, enriched, and structured. This entire process is called feature engineering.

Feature Engineering in the ML Pipeline

```
Raw Data
   ↓
Data Understanding / Description
   ↓
Exploratory Data Analysis (EDA)
   ↓
Feature Engineering      ← This entire stage
   ↓
   ├── 1. Feature Transformation
   ├── 2. Feature Construction
   ├── 3. Feature Selection
   ├── 4. Feature Extraction
   ↓
Model Training
   ↓
Evaluation & Tuning
   ↓
Deployment
```

Feature scaling and feature selection are sub-tasks that fall under the broader umbrella of feature engineering.

The 4 Types of Feature Engineering

1. Missing Value Imputation

Most ML models cannot handle missing values due to data collection errors, sensor failures, or incomplete records.

Why it matters: Mean/median imputation, regression based imputation, or simply dropping rows can preserve statistical relationships and prevent bias.

Options for handling missing values:

Strategy	When to Use
Drop rows	When missing values are very few (< 1–2%) and random
Drop column	When a feature has > 70–80% missing values
Mean/median imputation	Numerical data with no significant outliers
Mode imputation	For numerical data with outliers or skewed distribution
Forward fill / Backward fill	For time-series data
KNN imputation	When relationships between features can predict missing values
Model-based imputation	Train a regression/classification model to predict the missing values

2. Categorical Encoding

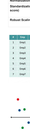
Machine learning models can understand only numbers — they work with numbers, not text labels. Categorical columns (like City, Gender, Product Type) must be converted to numerical representation.

Types of categorical variables:

- **Nominal** — no inherent order (e.g. Red, Blue, Green)
- **Ordinal** — has a meaningful order (e.g. Low, Medium, High)

Encoding techniques:

Technique	Best For	Notes
Label Encoding	Ordinal data	Assigns integer 0, 1, 2, …
One-Hot Encoding	Nominal categories (no ordinality)	Creates a binary column per category; can cause the curse of dimensionality
Ordinal Encoding	Ordinal categories	Manual mapping that preserves order



3.1 Outlier Detection

An **outlier** is a data point that deviates significantly from the rest of the distribution. Outliers can distort statistical measures (mean, variance) and degrade model performance, especially for linear models.

3.1.1 Mean without the Outlier

Example again:

2, 4, 5, 6, 7, 8, 9, 10

Mean without outlier

Mean = (Sum of elements) / (Number of values)

Sum = 54

Count = 8

Mean = 54 / 8 = 6.75

So the **average age is 6 years.**

3.1.2 Mean with the Outlier (60 year-old)

Now include the **60 year-old man.**

Data:

2, 4, 5, 6, 7, 8, 9, 10, 60

Sum = 111

Count = 9

Mean = 111 / 9 = **12.4**

3.1.3 Impact of the Outlier

Case	Mean
Without outlier	6.75
With outlier (60)	12.4

Detection methods:

Method	How It Works
Z-Score	Flags points beyond standard deviations from the mean
IQR Method	Flags below Q1 − 1.5×IQR or above Q3 + 1.5×IQR
Box Plot	Visual representation of whisker boundaries

Treatment strategies:

Strategy	Description
Cap / Clip	Replace outlier values with boundary values
Log transform	Compresses extreme values
Robust Scaler	Apply logarithmic transformation to compress the range
Keep	In some domains (fraud, medical), outliers ARE the signal — do not remove

3.1.4 Feature Scaling

When features have vastly different numerical ranges (e.g., age: 0–100 vs. salary: 10,000–100,000), raw values bias models toward the larger-scale features.

Rule: Fit scalers on training data only, then apply the same transformation to test data.

Feature scaling ensures all features contribute **proportionally**.

Scaling techniques:

Technique	Formula	Output Range	Best For
Min-Max Normalization	$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$	Bounded (0 to 1)	When distribution is not Gaussian; neural networks
Standardization (Z-score Normalization)	$x' = \frac{x - \mu}{\sigma}$	Unbounded (typically −3 to +3)	Most ML models; algorithms assuming Gaussian; SVM, logistic regression
Robust Scaling	$x' = \frac{x - \text{median}}{\text{IQR}}$	Centered on median	When data has significant outliers

Important: Models that **do not** need scaling: Decision Trees, Random Forest, XGBoost (tree-based models are invariant to monotonic transformations).

Models that **do need** scaling: Linear Regression, Logistic Regression, SVM, KNN, Neural Networks, PCA.

2. Feature Construction

Feature construction means **creating entirely new features** that do not exist in the raw dataset, by combining existing features or extracting new information from dates, text, or categorical variables.

This is the most creative aspect of feature engineering and depends heavily on **domain expertise** and **intuition**.

3. Feature Selection

Feature selection is the process of **identifying and keeping only the most relevant features** while discarding those that are redundant, irrelevant, or harmful to model performance.

Why features matter: Garbage in, garbage out. Irrelevant features introduce noise, increase training time, and can mislead the model.

Why Feature Selection Matters

- Reduces **overfitting** by removing noise variables
- Improves **model interpretability** (fewer features = simpler model)
- Cuts **training time** and computational cost
- Can improve **generalization** (test set performance)

Methods For Feature Selection

Method	Mechanism	Best For
Filter Methods	Score each feature by statistical measure (correlation, mutual information, chi-square)	Quick baseline; high-dimensional data
Wrapper Methods	Feature combination based on performance (e.g. RFE, forward selection, backward elimination)	Small feature sets; when accuracy is critical
Embedded Methods	Feature selection is built into the model training itself (e.g. Lasso, Decision Tree, Random Forest)	Efficient alternative to wrapper methods

4. Feature Extraction

Feature extraction creates **new transformed features** from the existing ones — often reducing the number of dimensions while retaining as much information as possible. Unlike feature selection (which keeps a subset of original features), extraction uses mathematical algorithms to find new representations of the data.

4.1 Principal Component Analysis (PCA)

PCA is an **unsupervised** linear dimensionality reduction technique. It finds the directions (principal components) of maximum variance in the data and projects it into a lower-dimensional subspace.

How it works:

1. Standardize the data
2. Compute the covariance matrix
3. Compute eigenvalues and eigenvectors
4. Sort eigenvalues by descending magnitude
5. Keep top k eigenvectors — these are the principal components
6. Project data onto the new k-dimensional space

$$X' = X \cdot W_k$$

Where W_k is the matrix of the top k eigenvectors.

Key properties:

- Principal components are **orthogonal** (uncorrelated)
- Order matters: PC1 captures the most variance, PC2 captures the next most, etc.
- **Significant variance ratio** tells you how much information each component carries
- Information loss is measurable — choose k to retain 95% explained variance

Best for: High-dimensional data (images, text embeddings, sensor arrays), removing multicollinearity, visualization (reduce to 2D/3D).

Limitation: Components are linear combinations of original features — **not interpretable**.

4.2 Linear Discriminant Analysis (LDA)

LDA is a **supervised** dimensionality reduction technique that finds the axes that maximize class separability rather than variance.

Objective: Find projections that maximize:

$$\frac{\text{Between-class variance}}{\text{Within-class variance}}$$

Where:

- S_B = Between-class scatter matrix
- S_W = Within-class scatter matrix

Key difference from PCA:

	PCA	LDA
Type	Unsupervised	Supervised
Goal	Maximize variance	Maximize class separability
Max components	min(n_samples, n_features) − 1	n_classes − 1
Assumes	Linear structure	Gaussian classes, equal covariance

Best for: Classification problems where class discrimination is important.

Summary Comparison

Type	Input	Output	Consequence	Pro
Transformation	Existing features	Modified same features	Loses interpretability	Cleaning, scaling, encoding raw data
Construction	Existing features	New additional features	Adds dimensionality	Adding meaningful domain knowledge
Selection	Subset of features	Same features, fewer rows	Loses information	Removing noise, improving efficiency
Extraction	Raw data	New abstract features	Low interpretability	Handling unstructured data, maximum compression

Common Feature Engineering Mistakes

Mistake	Consequence	Fix
Scaling before train/test split	Data leakage from test set into training	Fit scaler on train set only, transform both
Imputing before split	Same data leakage issue	Split first, then impute on train only
One-hot encoding without dropping one column	Multicollinearity (dummy variable trap)	Use `drop_first=True`
Removing outliers blindly	May delete genuine rare events	Investigate first, then decide
Applying PCA without feature selection	PCA uses every feature — noisy features corrupt components	Select relevant features first

Practical Workflow

```
1. Understand the data (EDA)
   ↓
2. Handle missing values
   ↓
3. Encode categorical variables
   ↓
4. Detect and handle outliers
   ↓
5. Construct new features (domain knowledge)
   ↓
6. Scale numerical features
   ↓
7. Select relevant features
   ↓
8. Extract compressed representations (PCA, LDA)
   ↓
9. Feed into ML model
```